

Control of Flow with TSQL

By Don Schlichting

Introduction

Transact SQL statements are usually executed in sequential order. When a long stored procedure or statement is created, line 1 will run, followed by line 2, followed by line 3. In many cases, we will want to interrupt this normal flow. Special processing may be needed depending on the results just received. We may want to branch or loop for example. Like other programming languages, Transact SQL has key words for controlling this order of execution. Collectively, these key words are called a Control of Flow language. This article examines these key words and how they are used to interrelate and connect separate statements into a traditional programming like structure.

IF

One of the most often used Control of Flow key words is IF. Like other programming languages, IF is used to test a Boolean condition. When the result of the IF statement is True, the next proceeding line is executed. From inside Query Analyzer, change to the pubs database and execute:

```
IF EXISTS(SELECT au_fname FROM authors WHERE au_lname = 'Smith')  
    PRINT 'Found it'
```

The words "Found it" should be returned.

ELSE

The ELSE key word can be coupled with IF to execute logic when the IF statement returns False. Else is an optional statement.

```
IF EXISTS(SELECT au_fname FROM authors WHERE au_lname = Jones)  
    PRINT 'Found it'  
ELSE  
    PRINT 'Not Found'
```

This statement will return "Not Found." Unlike other languages, TSQL does not have an ELSE IF key word. The work around is to nest IF ELSE statements inside other IF ELSE statements. There are no limits to the number of times an IF ELSE statement can be nested. There is no END IF statement.

```
IF EXISTS(SELECT au_fname FROM authors WHERE au_lname = 'Jones')  
    PRINT 'Found it Jones'
```

```

ELSE
    IF EXISTS(SELECT au_fname FROM authors WHERE au_lname = 'Smith')
        PRINT 'Found it Smith'

    ELSE
        PRINT 'Not Found Smith or Jones'

```

BEGIN...END

The BEGIN and END keywords are used to group multiple lines into one Statement Block. An example of when Statement Blocks are required is in the result of an IF ELSE statement. In this example, two PRINT lines are wanted on True result. Nothing should be returned on a False result.

```

IF EXISTS(SELECT au_fname FROM authors WHERE au_lname = 'Jones')
    PRINT 'Found it'
    PRINT 'Last name of Jones'

```

The statement will return:

Last name of Jones

Even though there is no Jones in the database. The unwanted return is generated because IF statements only return the next line or Statement Block. In our case, there is no Statement Block, because the BEGIN and END key words are not used, so only the next line of: PRINT 'Found it' is considered to be tied to the IF statement. The PRINT 'Last name of Jones' is viewed as a new command all by itself--not attached to the IF. To get the desired results, modify the statement to:

```

IF EXISTS(SELECT au_fname FROM authors WHERE au_lname = 'Jones')
    BEGIN
        PRINT 'Found it'
        PRINT 'Last name of Jones'
    END

```

Now the statement will come back without any of the PRINT lines being executed.

GOTO

Yes, the old faithful GOTO, despised as the spaghetti code instigator in other languages, is alive and well in TSQL. GOTO simply defines a label, and lets the code jump to that label from some other point. Cursors usually make use of a GOTO statement. However, a GOTO can also be used alone. This statement defines a GOTO Label, do_it_again, by ending it with a colon. When the second IF is executed, GOTO sends control back to the Label if the count is less than 100.

```

DECLARE @Count int
SET @Count = 0

```

do_it_again:

```

IF @Count < 100
    BEGIN
        PRINT 'Hello World'
        SET @Count = @Count + 1
    END

```

```

IF @Count < 100
    GOTO do_it_again

```

A point sometimes overlooked is that any statement ending in a colon is considered a label. In addition, the code under the label is executed on the first pass. In other words, the code logically belonging to a label is executed as standard TSQL when first encountered. So the value of @Count, when it reaches:

```

IF @Count < 100
    GOTO do_it_again

```

For the first time is 1, not 0.

RETURN

When the RETURN key word is encountered, statement execution ends, unconditionally. Any lines following a RETURN are not executed. Optionally, the RETURN can pass an integer. If a RETURN is placed inside our Label code, only one "Hello World" will be returned.

```

DECLARE @Count int
SET @Count = 0

```

```

do_it_again:
    IF @Count < 100
        BEGIN
            PRINT 'Hello World'
            RETURN
            SET @Count = @Count + 1
        END

```

```

IF @Count < 100
    GOTO do_it_again

```

WAITFOR

WAITFOR allows statement execution to be paused for a delayed time amount, or until a specific time of day. This is similar to the PAUSE key word in other languages. The following example replaces the RETURN statement with a two second delay. Execution time will now take twenty seconds for a 10 count loop.

```

DECLARE @Count int
SET @Count = 0

```

```

do_it_again:
    IF @Count < 10

```

```

BEGIN
    PRINT 'Hello World'
    WAITFOR DELAY '00:00:02'
    SET @Count = @Count + 1
END

```

```

IF @Count < 10
    GOTO do_it_again

```

WHILE

Like other languages, the TSQL WHILE creates a loop that keeps executing until a Boolean False condition is received. Like IF statements, WHILE statements can be nested inside other WHILE statements. BEGIN and END are also used in the same IF statement way.

```

DECLARE @Count int
SET @Count = 0

WHILE @Count < 100
BEGIN
    PRINT 'Hello World'
    SET @Count = @Count + 1
END

```

BREAK...CONTINUE

BREAK and CONTINUE are used to exit, or continue executing WHILE or IF statements. The above statements have been modified to show an example of their use. The statement will only return ten rows now.

```

DECLARE @Count int
SET @Count = 0

WHILE @Count < 100
BEGIN
    PRINT 'Hello World'
    SET @Count = @Count + 1
    IF @Count > 10
        BREAK
    ELSE
        CONTINUE
END

```

Conclusion

Missing or misused BEGIN and END statements seem to cause most of the problems associated with flow. Another problem is that Query Analyzer does not indent flow statements well, leading to confusion as to where one statement block ends and another begins. However, used well, Control of Flow key words can transform Transact SQL statements into powerful programming structures.